```
$dom = Dom\HTMLDocument::createFromString($validHtml);
$xpath = new Dom\XPath($dom);

$queryOne = $xpath->query('//*[@id="first_para"]');
$queryTwo = $xpath->query('//*[@class="another_para"]');

print "Query returns instances of " . get_class($queryOne) . PHP_EOL;
print "... which contains instances of " . get_class($queryOne[0]) .
PHP_EOL;

Output:
Query returns instances of Dom\NodeList
... which contains instances of Dom\HTMLElement
```

The `Dom\XPath` class implements the *XML Path Language*, which we will get to in a moment. For now, just accept that `Dom\XPath::query('//*[id="first_para"]');` translates to "start at the root element of the document and return all elements that have an `id` attribute of 'first_para'". The `query()` method returns (surprise!) a `Dom\NodeList`, which contains `DOM\HTMLElement` objects representing each matching element in the DOM. `class` and `id` values can be accessed on these objects directly (no need to go through the entire `attributes` list):

```
var_dump($queryOne[0]->id);
var_dump($queryTwo[0]->className);
var_dump($queryTwo[1]->className);

Output:
string(10) "first_para"
string(12) "another_para"
string(12) "another_para"
```

XPath

The last example showed the basics of using the `Dom\XPath` class, which allows you to use the powerful XPath expression syntax to quickly find elements in an XML document (and therefore an HTML document). For quick traversal of the DOM, it is your go-to tool; it will enable you to access nodes deep within a DOM hierarchy with a tiny handful of code. Quick, concise DOM navigation will make for far more maintainable code when